Palouse Alliance Community Resource Platform

Project Testing and Acceptance Plan

Palouse Alliance Team

05-SP26-FA26-PAHIFC-WEBMOB

Omar Adil, Vijay Deva, Kaeden Peterson, Aryan Puthran

April 2026

Table of Contents

I. Introduction **5**
- I.1. Project Introduction 5
- I.2. Background and Related Work 5
- I.3. Project Overview 5
- I.4. Client and Stakeholder Identification and Preferences 6

II. Team Member Introduction **6**
- II.1. Omar Adil 6
- II.2. Vijay Deva 6
- II.3. Kaeden Peterson 7
- II.4. Aryan Puthran 7

III. System Requirements Specification **7**
- III.1. Functional Requirements 7
  - III.1.1. Search and Discovery 8
  - III.1.2. Events and Calendar 8
  - III.1.3. User Management 8
  - III.1.4. Content Management 9
  - III.1.5. Information Display and Privacy 9
  - III.1.6. Volunteer and Community Features 10
  - III.1.7. Platform Support and Analytics 10
- III.2. Non-Functional Requirements 11
- III.3. Use Cases 12
  - III.3.1. [UC-1] Search and Attend Events 12
  - III.3.2. [UC-2] Upload and Approve Event or Flyer 13
  - III.3.3. [UC-3] Manage Organization Listing 13
  - III.3.4. [UC-4] Volunteer Opportunity Discovery 14
  - III.3.5. [UC-5] Access Community Analytics 14
- III.4. User Stories 15
  - III.4.1. [US-1] User Login 15
  - III.4.2. [US-2] Search for Services or Events 15
  - III.4.3. [US-3] Category Filtering 15
  - III.4.4. [US-4] Post Event or Flyer 16
  - III.4.5. [US-5] Admin Approves Submissions 16
  - III.4.6. [US-6] View Flyers and Meetings 16
  - III.4.7. [US-7] Manage Listing & Contact Info 16
- III.5. Traceability Matrix 17
- III.6. Standards 19
  - III.6.1. Standards Table 19
  - III.6.2. How These Standards are Applied 20
    - Requirements and Lifecycle 21
    - Accessibility 21

Security and Privacy 21
Web Standards and SEO 21
III.6.3. Scope and Compliance 21
IV. System Evolution 22
V. Architecture Design 22
V.1. System Overview 22
V.1.1. Layers / Major Components 22
V.1.2. Responsibility of Each Layer 23
V.2. Subsystem Decomposition 23
V.2.1. Search and Discovery Subsystem 23
V.2.2. Events and Calendar Subsystem 24
V.2.3. User Management Subsystem 24
V.2.4. Content Management Subsystem 25
V.2.5. Volunteer & Analytics Subsystem 25
V.2.6. Subsystem Services Overview 26
VI. Data Design 27
VI.1. Table and Field Overview 27
VI.1.1. Internal Users Table (auth.users) 27
VI.1.2. Public Users Table (public.users) 27
VI.1.3. Organizations Table (public.organizations) 27
VI.1.4. Events Table (public.events) 28
VI.1.5. Tags Table (public.tags) 28
VI.1.6. Event-Tags Table (public.event_tags) 29
VI.1.7. Categories Table (public.categories) 29
VI.2. Entity-Relationship Diagram 29
VII. User Interface Design 30
VII.1. Home Page 30
VII.2. Login Page 31
VII.3. Register Page 32
VII.4. Post Event Page 33
VII.5. Dashboard Page 33
VII.6. Admin Page 34
VII.7. Organizations Page 34
VIII. Constraints 35
IX. Project Testing and Acceptance Plan 36
IX.1. Overview 36
IX.1.1. Test Objectives and Schedule 36
IX.1.2. Scope 36
IX.2. Testing Strategy 37
IX.3. Test Plans 37
IX.3.1. Unit Testing 38

IX.3.2. Integration Testing 38
IX.3.3. System Testing 38
Functional Testing 39
Performance Testing 39
Standards and Constraints Verification / Testing 40
User Acceptance Testing 40
IX.4. Environment Requirements 40
Glossary 42
References 43
Appendix 44
A. Additional Use Cases 44
A.1. [UC-6] Manage Deployment and Branding 44
B. Additional User Stories 44
B.1. [US-8] Update Platform Configuration 44
B.2. [US-9] Discover Volunteer Opportunities 44
B.3. [US-10] View Platform Analytics 45

I. Introduction

I.1. Project Introduction

The Palouse Alliance Community Resource Platform is a digital initiative designed to make community health and wellness resources easier to find and easier to use across the Palouse region. Residents often need help quickly, but services are frequently scattered across separate nonprofit sites, county and state pages, flyers, and social media posts. That fragmentation creates real friction. People may not know what exists, which organization to contact, or whether information is still current. This platform addresses that gap by organizing trusted resources in one place and presenting them through a clear, searchable interface.

The platform is intended to support a wide range of users, including students, families, seniors, veterans, and individuals seeking housing support, food assistance, mental health services, or community programs. Rather than forcing residents to learn multiple websites and terminology, the platform will provide consistent categories, filters, and plain language descriptions. It will also highlight upcoming events and deadlines through a shared community calendar so residents can plan ahead instead of finding information at the last minute.

Accessibility is a core goal [3]. The project will follow established accessibility guidance [1], [3], [5] so that residents using assistive technologies, mobile devices, or limited bandwidth can still access critical information. In addition, the platform supports the organizations behind the services by reducing repeated outreach work and enabling simpler updates. Over time, this creates a more reliable community information hub, improves coordination among participating groups, and reduces the information gaps that can prevent people from getting help.

I.2. Background and Related Work

Currently, information about local services is spread across dozens of different websites and social media accounts, making it difficult for residents to find what they need. Many organizations are available to help, yet it is nearly impossible for residents to access the full range of services in a single search. This project is inspired by the Whiteman County Veterans website (whitmancountyveterans.com), which brings together vital information for local veterans. Research on community resource platforms highlights the importance of good design and accessibility [1-4].

Our platform will utilize that successful model to serve the entire Palouse community in hopes that all residents, not just veterans, have a single, reliable place to find the support they need. Past successes highlight the value of having an organization manage content updates, which helped to inform this project's design. We will deliver a custom made platform featuring a user friendly interface for searching, community calendars, and posting.
As the project develops, skills required include modern full stack web development, user authentication and data security, **SQL**, **AWS**, UI/UX design, database design, and content moderation and posting systems.

I.3. Project Overview

The Palouse Alliance Community Resource Platform will offer a community calendar and tools for organizations to post flyers for events, and meeting dates. Some core requirements include accessibility and responsiveness, the site should be available on laptop, tablet, and mobile devices, with support for those with disabilities. Members of the Alliance can keep their listing

current and share announcements. Visitors will be able to filter to make it easier to connect with the relevant event hosts. The calendar shows upcoming deadlines and events, while admins can moderate what is being posted to ensure the content is accurate and appropriate for the community. Technical development will rely on modern web **HTML** frameworks [4]. By focusing on making the website easy to use for the public and members, the project aims to improve events and lower barriers for everyone.

We expect to see increased discoverability and accessibility, better coordination, and a scalable platform that can be extended or adapted to future needs. We also aim to reduce the manpower required to post events and meeting dates so members can better focus on the community.

I.4. Client and Stakeholder Identification and Preferences

Allison Rockwell (allison.rockwell@wsu.edu) serves as the primary client. She is responsible for communicating requirements, reviewing deliverables, and providing consolidated feedback from Alliance membership. She requests a platform with a low learning curve for end users and contributors. The main stakeholders include the Alliance organizations, the community members accessing the website, event organizers and volunteers, and technical maintainers.

For members, the ability to update listings, share events, and post flyers is essential. They can also protect sensitive details and display only chosen contact or service information. The general public values reliable information, responsive design, and clear categorization.

The initial meeting from the stakeholder helped us to realize we need an integrated calendar and simple flyer posting options. Consistent branding are priorities due to the nature of the website (for the public). Communication between the development team and Alliance will be through regular biweekly meetings. The reviews will be to review progress with requirements and gather feedback.

The platform must be highly accessible, and have a simple interface, must be available on laptop, tablet, and mobile devices, must be secure and respect the privacy of organization members, align with community values such as being inclusive and supportive, and be low maintenance, with content management having intuitive interfaces [3-6].

II. Team Member Introduction

II.1. Omar Adil

Omar Adil is a Computer Science student at Washington State University, graduating in December 2026. As a Software Engineer Intern at Silk, he designed a modular session management framework in Python for secure communication with Silk Virtual SAN platforms, and built a dynamic API construction engine for RESTful requests and authentication. Omar has experience developing reusable API helpers and automating system validation, supporting scalable backend solutions. Proficient in Python, C/C++, and C#. He is skilled with PostgreSQL, Node.js, and DevOps tools like Docker and GitHub. For the project, Omar will focus on backend integration, API development, and secure session management.

II.2. Vijay Deva

Vijay Deva is a Computer Science major at Washington State University, expecting to graduate in December 2026. He previously interned at People Tech Group, where he developed and deployed a parking space detection machine learning pipeline using Yolo-NAS, achieving 97.7% accuracy, and integrated real-time model inference into a cloud-based management system. Vijay also built React and AngularJS UI components and implemented InfluxDB-powered data visualizations. He is adept with Python, C#, JavaScript frameworks, PostgreSQL, and Docker. For the project, Vijay will contribute to machine learning integration, full stack development, and deployment of analytical dashboards.

II.3. Kaeden Peterson

Kaeden Peterson is pursuing a Computer Science degree at Washington State University, class of December 2026. As an EECS Teaching Assistant, he supported courses by clarifying concepts, grading assignments, and ensuring consistent evaluation standards. Skilled in Python, C, C++, C#, and Java, Kaeden is also proficient with Docker, Linux/Bash, and collaborative tools like Git and GitHub. His experience coordinating with other TAs and proctoring exams allows him to manage technical and team responsibilities efficiently. For the team, Kaeden will lead testing, automation, and continuous integration setup, as well as contribute to core backend development.

II.4. Aryan Puthran

Aryan Puthran is a Computer Science student at Washington State University, graduating December 2026. As a VCEA Teaching Assistant, he supported over 200 students across programming and software engineering courses, offering debugging help, grading assignments, and facilitating group discussions to reinforce key concepts. During his Data Engineering internship at T-Mobile, he developed Grafana dashboards, analyzed network and location data, automated visualizations, and

migrated legacy automation scripts to Python to improve maintainability. Familiar with C++, Python, **SQL**, Grafana, and CI/CD tools, Aryan will contribute to data analytics, visualization, and DevOps processes for the project.

III. System Requirements Specification

The Requirements and Specification section details the functional and non-functional requirements for this project. Functional requirements cover essential features like displaying events, membership, posting content, and searching. Non-functional requirements outline system constraints related to usability, reliability, branding, and maintainability. This section also includes **use cases**, which aligns requirements with objectives for development of the application.

III.1. Functional Requirements

Functional requirements define the specific features, actions, and capabilities the platform must provide to its users. These requirements describe how users interact with the system and what tasks they can accomplish.

III.1.1. Search and Discovery

Functional Requirement	[FR-1] Search for Services and Events
Description	The system will provide a search bar enabling users to input keywords and find relevant services or events.
Source	Initial functional requirements, meeting notes.
Priority	Level 0 (Essential functionality)
Functional Requirement	[FR-2] Category-Based Filtering
Description	The system will allow users to filter services or events by selecting categories (e.g. Housing, Health) and/or by event-specific tags.
Source	Initial functional requirements, meeting notes.
Priority	Level 0 (Essential functionality)

III.1.2. Events and Calendar

Functional Requirement	[FR-3] Event Calendar Integration
Description	The platform will provide a public calendar which displays all approved events. Events can be added or edited (posting/editing only by members; approval required before publication).
Source	Initial functional requirements, meeting notes.
Priority	Level 0 (Essential functionality)
Functional	[FR-6] View Flyers and Meetings

Requirement	
Description	Users (including public) can view flyers, events, and upcoming meetings from the calendar.
Source	Initial functional requirements, meeting notes.
Priority	Level 0 (Essential functionality)

III.1.3. User Management

Functional Requirement	[FR-4] Member and Admin Account Management
Description	The system will support membership registration ("click here to become a member") and sign in abilities. Member and admin accounts will be verified upon creation.
Source	Meeting notes.
Priority	Level 0 (Essential functionality)

III.1.4. Content Management

Functional Requirement	[FR-5] Event and Flyer Upload and Approval
Description	Members can upload events and flyers (as easy as sending an email). All posts require an admin's approval before becoming public.
Source	Meeting notes.
Priority	Level 0 (Essential functionality)
Functional Requirement	[FR-7] Organization and Agency Listings
Description	The system will include a dedicated page to listing community agencies/members.
Source	Meeting notes.
Priority	Level 1 (Desirable functionality)

III.1.5. Information Display and Privacy

Functional Requirement	[FR-8] Display Contact Information
Description	Listings in the calendar will show relevant contact information, available to all

	users (including public).
Source	Initial functional requirements.
Priority	Level 0 (Essential functionality)
Functional Requirement	[FR-9] Selective Display of Personal Information
Description	The platform will allow members to choose which personal information to share with optional fields and a disclaimer during registration and posting.
Source	Meeting notes.
Priority	Level 1 (Desirable functionality)

III.1.6. Volunteer and Community Features

Functional Requirement	[FR-10] **GivePulse** Integration
Description	The site will include a dedicated section linking **GivePulse** to improve volunteer activities.
Source	Meeting notes.
Priority	Level 0 (Essential functionality)

III.1.7. Platform Support and Analytics

Functional Requirement	[FR-11] Analytics and Attendance Tracking
Description	The system will track and display basic analytics (e.g. registration counts, attendance), available to admin users.
Source	Meeting notes.
Priority	Level 2 (Optional functionality)
Functional Requirement	[FR-12] Custom Domain Name Support
Description	The platform will support deployment under a custom domain name.
Source	Meeting notes.
Priority	Level 1 (Desirable functionality)
Functional	[FR-13] Branding and Theming

Requirement	
Description	The website will use logos, palettes, and branding guidelines provided by the stakeholders.
Source	Meeting notes.
Priority	Level 1 (Desirable functionality)
Functional Requirement	[FR-14] Responsive Web Design
Description	The platform will be responsive for laptop, tablet, and mobile displays.
Source	Meeting notes.
Priority	Level 0 (Essential functionality)
Functional Requirement	[FR-15] Search Engine Display
Description	Website pages will be visible in web search engines.
Source	Meeting notes.
Priority	Level 2 (Optional functionality)
Functional Requirement	[FR-16] Email Notifications for New Resources
Description	The system will send an email notification to all members when a new resource is posted on the website.
Source	Email from Chris.
Priority	Level 0 (Essential functionality)

III.2. Non-Functional Requirements

Non-functional requirements focus on the qualities and operating standards of the platform rather than its direct functionalities. They specify how the system performs under certain conditions or constraints and help ensure a quality user experience.

Non-Functional Requirement	Description
[NFR-1] Usability	The interface should be intuitive and accessible for all users [3]. Uploading content (events, flyers, etc.) should be as easy as sending an email. Accessibility standards are followed.

[NFR-2] Scalability	The platform must handle a growing number of users and events without a degradation in performance [4].
[NFR-3] Reliability	The platform should be available 99% of the time, with minimal planned downtime [3].
[NFR-4] Maintainability	The tech stack should be modular and documented. Code reviews and version control will be used.
[NFR-5] Security and Privacy	Sensitive user data will be stored securely [5][6]. Members have control over public information. Authentication will be used.
[NFR-6] Platform Compatibility	The system will be accessible on modern browsers and devices (laptops, tablets, and phones).
[NFR-7] Brand Consistency	The site will consistently apply provided branding, including logos and palettes.

III.3. Use Cases

This section describes typical scenarios in which users interact with the Palouse Alliance Community Resource Platform. **Use cases** clarify how actors achieve goals within the system and show the relationships between system requirements and the user experience. See *Figure 1* below for an overview of system actors and their main actions.

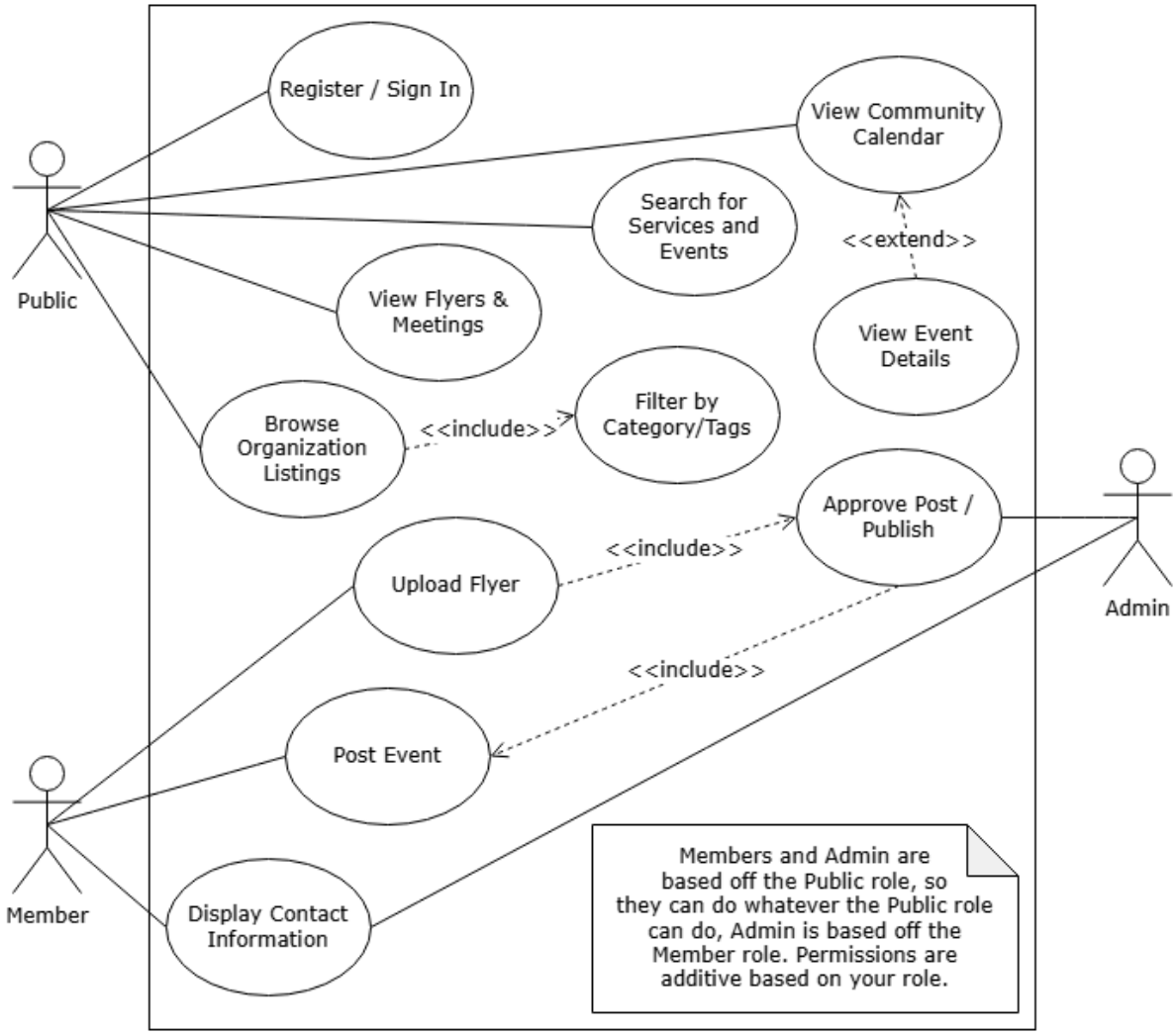

Figure 1: Use Case Diagram

III.3.1. [UC-1] Search and Attend Events

Use Case	Search and Attend Events
Actors	Public user, Member
Pre Condition	User is on the homepage (login not required).
Post Condition	User discovers events and may add to their calendar or explore details.
Main Flow	- User enters keywords in the search bar. - User optionally applies category or tag filters. - System displays a list of relevant services or events. - User selects an event to view more information. - User can save event details.

Alternative Flow	- No results found, user can adjust filters or submit a request for help.
Related Requirements	FR-1: Search for Services and Events FR-2: Category-Based Filtering FR-3: Event Calendar Integration FR-6: View Flyers and Meetings

III.3.2. [UC-2] Upload and Approve Event or Flyer

Use Case	Upload and Approve Event or Flyer
Actors	Member (uploads), Admin (approves)
Pre Condition	Member is logged in and accesses the event/flyer submission interface.
Post Condition	Event or flyer is posted to the calendar after admin approval.
Main Flow	- Member logs in and navigates to the submission page. - Member enters information and uploads files. - System sets item status as pending approval. - Admin logs in, reviews pending posts. - Admin approves (publishes) or rejects the submitted item.
Alternative Flow	- Admin rejects item, flyer/post isn't shown on public page.
Related Requirements	FR-4: Member and Admin Account Management FR-5: Event and Flyer Upload and Approval FR-16: Email Notifications for New Resources

III.3.3. [UC-3] Manage Organization Listing

Use Case	Manage Organization Listing
Actors	Member, Admin
Pre Condition	Member or admin is logged in and have access to their organization listing.
Post Condition	Organization listing is added, updated, or deleted.
Main Flow	- Member logs in and access the organization management page. - Member edits existing listings or create a new one (description, contacts, category, privacy). - Member saves changes. - System submits for admin approval.
Alternative Flow	- Admin removes listing, system archives it.

Related Requirements	FR-4: Member and Admin Account Management FR-7: Organization and Agency Listings FR-8: Display Contact Information FR-9: Display Contact Information

III.3.4. [UC-4] Volunteer Opportunity Discovery

Use Case	Volunteer Opportunity Discovery
Actors	Public user, Member
Pre Condition	User is on the homepage.
Post Condition	User discovers and explores volunteer opportunities via **GivePulse** integration.
Main Flow	- User navigates to the volunteer section. - System displays available opportunities via **GivePulse**. - User reviews details and may register or visit the external site.
Alternative Flow	- No current opportunities listed.
Related Requirements	FR-10: **GivePulse** Integration

III.3.5. [UC-5] Access Community Analytics

Use Case	Access Community Analytics
Actors	Admin
Pre Condition	Admin is logged in.
Post Condition	Admin views registration counts, attendance data, and other analytics.
Main Flow	- Admin logs into the dashboard. - Admin selects the analytics or reports section. - System displays up-to-date metrics and graphs.
Alternative Flow	- Data unavailable, system notifies admin of delay or error.
Related Requirements	FR-11: Analytics and Attendance Tracking

Please see the Appendix for additional **use cases**.

III.4. User Stories

This section presents user stories that show goals and interactions of the Palouse Alliance Community Resource Platform from different user perspectives. Each user story is in Gherkin syntax to better specify user intent, relevant context, actions taken, and desired outcomes.

III.4.1. [US-1] User Login

As a member,
I want to log in to my account
so that I can access member-only features and manage my listings.

Feature: User Login
Scenario: Valid member logs in
    Given the member is on the login page
    When they enter their correct username and password
    Then they should be redirected to their account dashboard

III.4.2. [US-2] Search for Services or Events

As a member,
I want to search for community services and events
so that I can find relevant support or activities quickly.

Feature: Service or Event Search
Scenario: User searches for a service or event
    Given the user is on the homepage
    When they enter a search term in the search bar
    Then relevant services or event listings should be displayed

III.4.3. [US-3] Category Filtering

As a visitor,
I want to filter services and events by category or tag
so that I can easily narrow down the results to what I need.

Feature: Category Filtering
Scenario: Filtering by category or tag
    Given the user is viewing services or events
    When they select a specific category or tag filter
    Then only relevant services or events should be shown

III.4.4. [US-4] Post Event or Flyer

As a member,
I want to submit a new event or flyer
so that our organization can share information with the community.

Feature: Event/Flyer Posting

Scenario: Member posts an event or flyer
Given the member is logged in and on the submission page
When they provide event or flyer details and submit
Then the submission should be sent for admin approval

III.4.5. [US-5] Admin Approves Submissions

As an admin,
I want to review and approve submitted events and flyers before publication
so that only appropriate content appears publicly.

Feature: Submission Approval
Scenario: Admin approves a new submission
Given the admin is reviewing pending submissions
When they approve a submission
Then the event or flyer should be published for public view

III.4.6. [US-6] View Flyers and Meetings

As a public user,
I want to view upcoming flyers, meetings, and events
so that I can stay informed about local resources and happenings.

Feature: Viewing Events
Scenario: User browses upcoming events
Given the user is on the calendar or events page
When they view the list of upcoming events
Then event details should be displayed

III.4.7. [US-7] Manage Listing & Contact Info

As a member,
I want to edit my organization's listing and select what contact information is shared publicly
so that I retain privacy and provide correct details.

Feature: Organization Listing Management
Scenario: Member edits listing and selects privacy options
Given the member is logged in and managing their listing
When they update details and set privacy preferences
Then the changes should be saved and appear accordingly

Please see the Appendix for additional user stories.

III.5. Traceability Matrix

Functional Requirement	Use Cases	User Stories	Priority

FR-1: Search for Services and Events	UC-1: Search and Attend Events	US1: As a member, I want to log in to my account	Level 0
FR-2: Category-Based Filtering	UC-1: Search and Attend Events	US2: As a resident, I want to search for community services and events US3: As a visitor, I want to filter services and events	Level 0
FR-3: Event Calendar Integration	UC-1: Search and Attend Events	US2: As a resident, I want to search for community services and events	Level 0
FR-4: Member and Admin Account Management	UC-2: Upload and Approve Event or Flyer UC-3: Manage Organization Listing	US1: As a member, I want to log in to my account US7: As a member, I want to edit my organization's listing	Level 0
FR-5: Event and Flyer Upload and Approval	UC-2: Upload and Approve Event or Flyer	US4: As a member, I want to submit a new event or flyer US5: As an admin, I want to review and approve submissions	Level 0
FR-6: View Flyers and Meetings	UC-1: Search and Attend Events	US6: As a public user, I want to view upcoming flyers, meetings, and events	Level 0
FR-7: Organization and Agency Listings	UC-3: Manage Organization Listing	US7: As a member, I want to edit my organization's listing	Level 1
FR-8: Display Contact Information	UC-3: Manage Organization Listing	US7: As a member, I want to edit my organization's listing	Level 0
FR-9: Selective Display of Personal Information	UC-3: Manage Organization Listing	US7: As a member, I want to edit my organization's listing and set privacy preferences	Level 0
FR-10: **GivePulse** Integration	UC-4: Volunteer Opportunity Discovery	US-9: Discover Volunteer Opportunities (Appendix)	Level 0
FR-11: Analytics and Attendance Tracking	UC-5: Access Community Analytics	US-10: View Platform Analytics (Appendix)	Level 2
FR-12: Custom Domain Name Support	UC-6: Manage Deployment and Branding (Appendix)	US-8: Update Platform Configuration (Appendix)	Level 1
FR-13: Branding and	UC-6: Manage	US-8: Update Platform	Level 1

Theming	Deployment and Branding (Appendix)	Configuration (Appendix)	
FR-14: Responsive Web Design	UC-1: Search and Attend Events	US6: As a public user, I want to view upcoming flyers, meetings, and events	Level 0
FR-15: Search Engine Display	UC-6: Manage Deployment and Branding (Appendix)	US-8: Update Platform Configuration (Appendix)	Level 2
FR-16: Email Notifications for New Resources	UC-2: Upload and Approve Event or Flyer	US5: As an admin, I want to review and approve submissions	Level 0

III.6. Standards

This section lists the technical, legal, and professional standards that inform our requirements and design choices. Each standard is mapped to the parts of the specification it affects, including functional requirements, non-functional requirements, design, testing, security, and accessibility.

III.6.1. Standards Table

Standard / Code	Domain	Description	Application in Project
IEEE 830-1998: Software Requirements Specification [8]	Software Documentation	Guidance on the structure and content of SRS documents.	Used to organize functional and non-functional requirements, **use cases**, user stories, and the traceability matrix. Helps ensure requirements are clear, testable, and verifiable across the SRS.
ISO/IEC/IEEE 12207:2017: Software Life Cycle Processes [9]	Project Management / Process	Defines lifecycle stages and roles.	Shapes project phases from requirements through maintenance, and supports change control in System Evolution.
IEEE 1016-2009: Software Design Description (SDD) [10]	Design Documentation	Structure for describing software architecture.	Guides design artifacts such as module descriptions and interfaces. Supports maintainability (NFR-4) and documentation of calendar, listing, authentication, and API components.
IEEE 829-2008: Test	QA / Testing	Formats for test plans, cases, and reports.	Used to document verification and validation for search,

Documentation [11]			posting and approval flows, calendar features, accessibility, and regression testing. Mapped from FR-1 to FR-6, and FR-14 and FR-16.
W3C **WCAG** 2.2 (2023): Web Content Accessibility Guidelines [12]	Web / UI Accessibility	Success criteria and techniques to make web content perceivable, operable, and understandable.	Applied to UI, forms, calendars, flyers, and listing pages. Covers text alternatives, keyboard navigation, and clear labels and error messages. Supports NFR-1 and FR-3, FR-6, and FR-14.
WAI-ARIA 1.2 (2021): Accessible Rich Internet Applications [13]	Web Accessibility	Properties for accessible content.	Used for interactive components such as calendar widgets, filters, dialogs, and status messages to improve screen reader and keyboard support. Supports NFR-1, FR-2, and FR-3.
OWASP ASVS v4.0.3 (2021): Application Security Verification Standard [14]	Security	Requirements for authentication, sessions, and input validation.	Applied to member and admin accounts, session handling, upload validation, CSRF protections, and secure logging. Supports FR-4, FR-5, FR-16, and NFR-5.
IETF RFC 8446 (2018): TLS 1.3 [15]	Security / Transport	Transport protocol for encrypting data.	Required for all network traffic over **HTTPS**, including authentication, uploads, admin dashboards, and API calls. Supports NFR-5.
GDPR: Regulation (EU) 2016/679 (enforced 2018) [16]	Data Protection / Privacy	Framework for processing personal data and their rights.	Informs data minimization, consent for contact details and notifications, and ability to access and delete. Supports FR-4, FR-9, FR-16, and NFR-5.
CAN-SPAM Act (2003) [17]	Email / Legal	U.S. requirements for commercial email and opt-out.	Applied to FR-16 email notifications, including accurate headers, clear identification, and unsubscribe or opt-out options.

III.6.2. How These Standards are Applied

Requirements and Lifecycle

- Requirements, **use cases**, and the traceability matrix follow IEEE 830 so each requirement is clear, and testable and traceable.
- The overall project lifecycle follows ISO/IEC/IEEE 12207, including roles, deliverables, and change management.
- Architecture and module documentation follow IEEE 1016 to support long-term maintainability (NFR-4).
- Test plans and test artifacts follow IEEE 829 to keep validation and regression testing consistent and auditable.

Accessibility

- Core user flows like search, filtering, calendars, flyer views, and submission forms will meet **WCAG** 2.2 success criteria.
- Interactive UI elements will use WAI-ARIA patterns where appropriate to ensure strong screen reader and keyboard support.
- U.S. accessibility guidance from Section 508 and the ADA will inform content clarity and effective communication.

Security and Privacy

- Authentication, authorization, session controls, and input validation will align with OWASP ASVS at a level appropriate for a public community platform (NFR-5).
- All traffic will be encrypted using **HTTPS**, with a preference for TLS 1.3 as defined in RFC 8446.
- Personal data handling will follow privacy-by-default practices inspired by GDPR, including consent and data minimization. Email notifications will follow CAN-SPAM rules.
- Controls for CSRF, XSS, and file upload validation will be implemented and captured in the security design.

Web Standards and SEO

- Semantic **HTML**, appropriate meta tags, and structured data will support accessibility and discoverability (FR-15).
- Branding and theming (FR-13) will be applied consistently without breaking semantic structure or assistive technology support.

III.6.3. Scope and Compliance

HIPAA is out of scope because the platform doesn’t collect or process protected health information (PHI). If future requirements introduce PHI, then HIPAA controls will be added and this section updated.

Where legal obligations vary by region (for example GDPR and CCPA), the project will use a risk-based approach. We will implement consent and data minimization by default, then add region-specific controls as needed.

IV. System Evolution

The Palouse Alliance Community Resource Platform is designed to be flexible and adaptable as technology and community needs change. We anticipate that user expectations will evolve, new types of events or resources may need to be supported, and integration with additional services or partner organizations may be required in the future. To address this, the system is built on **modular architecture** and cloud-based infrastructure, which can scale to accommodate increasing usage and new features.

Potential change points include updates to branding, accessibility standards, or security requirements, as well as shifts in local or regional partnerships. By documenting our code, maintaining modular interfaces, and following best practices for maintainability and accessibility, we aim to reduce risks associated with technological changes and ensure the platform remains relevant and reliable as user and organizational needs evolve.

V. Architecture Design

This section describes the high-level technical structure of the platform, including the rationale for architectural decisions, the major layers and subsystems, and how system responsibilities are divided to have maintainable and scalable development.

V.1. System Overview

The Palouse Alliance Community Resource Platform is designed as a web application that prioritizes accessibility and ease of use. The system consists of a responsive frontend, a RESTful backend, and a relational database, combined with integration points for third-party services (i.e. **GivePulse** for volunteer opportunities). It follows a layered architectural pattern that separates presentation, logic, data management, and authentication, supporting maintainability and security.

The system utilizes a client-server architecture with layered decomposition. The reason for this approach is that it allows for a separation of concerns (frontend is isolated from backend logic and data management), making it easier to scale, maintain, and update the system independently. The REST API further ensures that new interfaces or integration points can be added without disrupting the backend layer.

V.1.1. Layers / Major Components

- Frontend: React or similar framework (based on requirements, flexible for accessibility), communicates with backend APIs, performs client-side rendering of listings, events, flyers, calendar views, and handles user input and interaction.
- Backend: Node.js, Vite, processes business logic, content moderation, authentication, and manages data access.
- Database: Relational (PostgreSQL), stores events, organizations, users, tags, and other platform entities.
- External Integration: Support for **GivePulse** volunteer opportunities and email notification service.
- Admin Interface: Separate dashboard for review and approval of submissions and access to analytics.

V.1.2. Responsibility of Each Layer

- Frontend: User interaction, responsive displays, accessibility enforcement, form validation, navigation, and presentation logic.
- Backend: API endpoint processing, authentication, business logic, validation, moderation, and analytics computation.
- Database: Data persistence, query optimization, and integrity enforcement.
- Admin Interface: Approve or reject content, manage accounts, and view analytics.

V.2. Subsystem Decomposition

Explains how the system is divided into major subsystems, each with distinct responsibilities such as search, calendar management, user authentication, content posting, analytics, and volunteer integration. For each subsystem, the interface and main functionalities are described.

V.2.1. Search and Discovery Subsystem

a) Description: Handles user queries for searching services, events, and flyers. Supports keyword search, and category and tag filtering.
b) Concepts and Algorithms Generated:
   i) Keyword matching using **SQL** LIKE or full text search.
   ii) Category/tag filters applied as **SQL** joins or query modifiers.
c) Interface Description: Enables users to find relevant services, events, and flyers using keyword search, filters, and tags. Handles query logic and delivers results efficiently.

Services Provided

Service Name	Provided To	Description
SearchEventsAndServices	Frontend	Receives query, filter criteria, and returns matching queries.
ListCategoriesAndTags	Frontend	Filter UI population.

Services Required

Service	Service Provider
Access to events, organizations, and tags.	Supabase
Authentication if searching restricted or member only resources.	Supabase

V.2.2. Events and Calendar Subsystem

a) Description: Manages the public calendar, event creation/submission, and flyer display. Handles posting/editing by members and moderation by admins.
b) Concepts and Algorithms Generated:
   i) **CRUD** operations for events and flyers.
   ii) Status field for moderation workflow (pending, approved, rejected).
   iii) Calendar rendering (frontend), date/time formatting.
c) Interface Description: Manages event listings, flyers, and the public calendar. Supports posting, editing, moderation, and viewing of events, ensuring timely access to community information.

Services Provided

Service Name	Provided To	Description
ListCalendarEvents	Frontend, public users	Returns events and flyers.
SubmitEventOrFlyer	Members	Creates pending events and flyers.

ApproveEventOrFlyer	Admin dashboard	Updates events and flyer status.

Services Required

Service	Service Provider
Authentication for submission/approval.	Supabase
Persistence for storing event/flyer details.	Supabase

V.2.3. User Management Subsystem

a) Description: Handles member and admin registration, login, and account verification. Responsible for authentication, session management, and account profile privacy controls.
b) Concepts and Algorithms Generated:
   i) Secure password storage.
   ii) Role-based access (member, admin).
c) Interface Description: Controls member and admin registration, authentication, role assignment, and privacy options for account profiles and contact details.

Services Provided

Service Name	Provided To	Description
RegisterUser	Registration page	Creates user account.
LoginUser	Login page	Returns access token.

Services Required

Service	Service Provider
Database persistence for user records.	Supabase

V.2.4. Content Management Subsystem

a) Description: Enables members to post new events and flyers, and allows admins to approve or reject submissions. Supports organization/agency listings management.
b) Concepts and Algorithms Generated:
   i) Submission workflows: file upload, status management.
   ii) **CRUD** operations for organization listings.
   iii) Approval workflow triggers email notification on publishing.
c) Interface Description: Allows organizational members to post new events, flyers, and listings; features workflows for moderation, editing, and approval.

Services Provided

Service Name	Provided To	Description
SubmitContent	Members	Allows posting new events, flyers, or organization listings.
ApproveContent	Admin interface	For moderation.
UpdateListing	Members and admins	Allows them to update organizational info.

Services Required

Service	Service Provider
Database storage for events, flyers, organizations.	Supabase
File upload service.	Supabase
Email notifications for status updates (requirement FR-16).	N/A

V.2.5. Volunteer & Analytics Subsystem

a) Description: Integrates **GivePulse** for volunteer opportunities. Provides admin access to basic analytics.
b) Concepts and Algorithms Generated:
   i) External API calls to **GivePulse**.
   ii) Basic registration and attendance counters.
   iii) Analytics dashboard rendering.
c) Interface Description: Integrates external volunteer services (like **GivePulse**), presents analytics dashboards for admin users, and tracks platform usage and attendance.

Services Provided

Service Name	Provided To	Description
ShowVolunteerOpportunities	Public	Fetches and displays **GivePulse** data.
ShowAnalytics	Admin	Can view registration and attendance stats.

Services Required

Service	Service Provider
GivePulse API or embed access.	**GivePulse**

Database read for registration/activity counts.	Supabase

V.2.6. Subsystem Services Overview

The diagram below illustrates the platform's major subsystems, the interfaces (services) they provide and require, and their relationships.

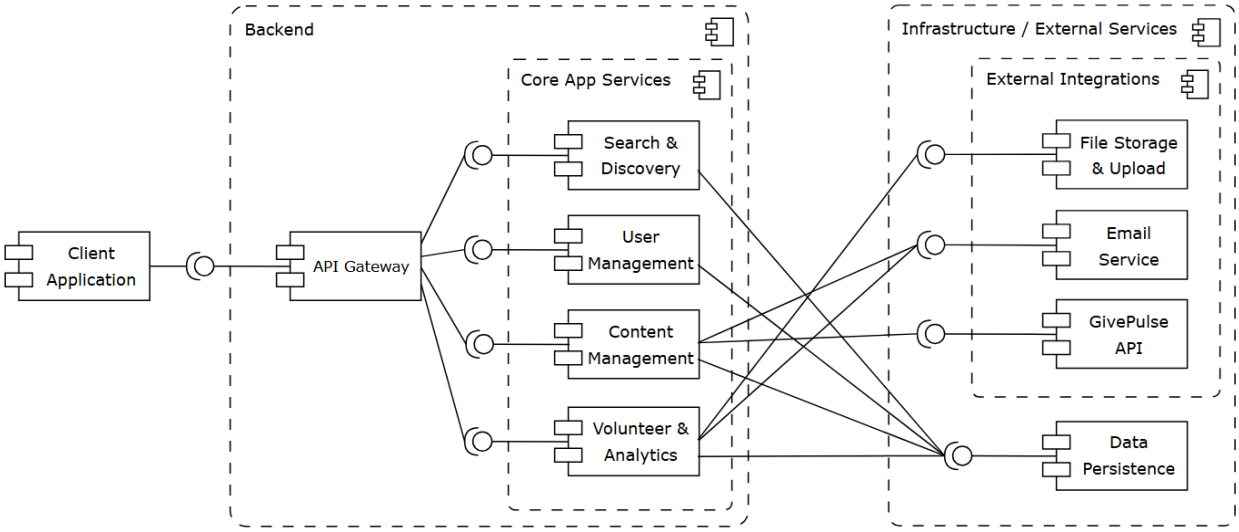

Figure 2: Ball and Socket Diagram

VI. Data Design

Details the structure of the platform's database, including tables, relationships, and the data entities that support the main features (users, organizations, events, tags, categories). Explains how data is stored and linked for reliability and performance.

VI.1. Table and Field Overview

Provides a summary of the fields and columns for each database table, reflecting the data needed to manage accounts, event listings, organizational information, and tagging.

VI.1.1. Internal Users Table (auth.users)

Stores authentication credentials for users of the platform. This table is managed by Supabase and handles secure storage of user email and password hashes. All platform users must have a corresponding record here.

Column	Type	Properties	Default Value
email	text	non-nullable	N/A
password	N/A	non-nullable	N/A

VI.1.2. Public Users Table (public.users)

Tracks individual users participating in the system, whether as members or admins. Links users to their affiliated organization and assigns their role within the platform.

Column	Type	Properties	Default Value
id	uuid	primary key, non-nullable	auth.uid()
role	user_role ("admin" or "member")	non-nullable	member
organization_id	uuid	foreign key, non-nullable	N/A

VI.1.3. Organizations Table (public.organizations)

Represents organizations or agencies that participate in the platform, such as community groups or service providers. Contains descriptive details and relevant contact info.

Column	Type	Properties	Default Value
id	uuid	primary key, non-nullable	gen_random_uuid()
name	text	non-nullable	N/A

description	text	nullable	N/A
phone_number	text	nullable	N/A
email	text	nullable	N/A
location	text	nullable	N/A

VI.1.4. Events Table (public.events)

Contains all events and flyers posted by organizations, including scheduling, description, and approval tracking. Events are tied to organizations, can have categories, and are associated with tags.

Column	Type	Properties	Default Value
id	uuid	primary key, non-nullable	gen_random_uuid()
organization_id	uuid	foreign key, non-nullable	N/A
category_id	uuid	foreign key, non-nullable	N/A
status	event_status (“pending”, “approved”, “rejected”)	non-nullable	pending
title	text	non-nullable	N/A
description	text	nullable	N/A
start_datetime	timestampz	non-nullable	N/A
end_datetime	timestampz	non-nullable	N/A
location	text	nullable	N/A
volunteer_url	text	nullable	N/A
created_by	uuid	non-nullable	N/A

VI.1.5. Tags Table (public.tags)

Provides categorization tags for events, allowing users to filter and search by topic.

Column	Type	Properties	Default Value

id	uuid	primary key, non-nullable	gen_random_uuid()
name	text	non-nullable	N/A

VI.1.6. Event-Tags Table (public.event_tags)

Associates tags with events using a many-to-many relationship. Each entry connects an event to a tag, enabling flexible filtering and searching.

Column	Type	Properties	Default Value
event_id	uuid	composite key, foreign key, non-nullable	N/A
tag_id	uuid	composite key, foreign key, non-nullable	N/A

VI.1.7. Categories Table (public.categories)

Defines broad categories for organizations and events. Used for filtering and browsing.

Column	Type	Properties	Default Value
id	uuid	primary key, non-nullable	gen_random_uuid()
name	text	non-nullable	N/A

VI.2. Entity-Relationship Diagram

Describes how the main entities relate to one another, outlining the connections between users, organizations, events, and tags for consistent, scalable data management.

For symbols, || signifies one and exactly one, while ○∈ signifies a zero or many relationship.

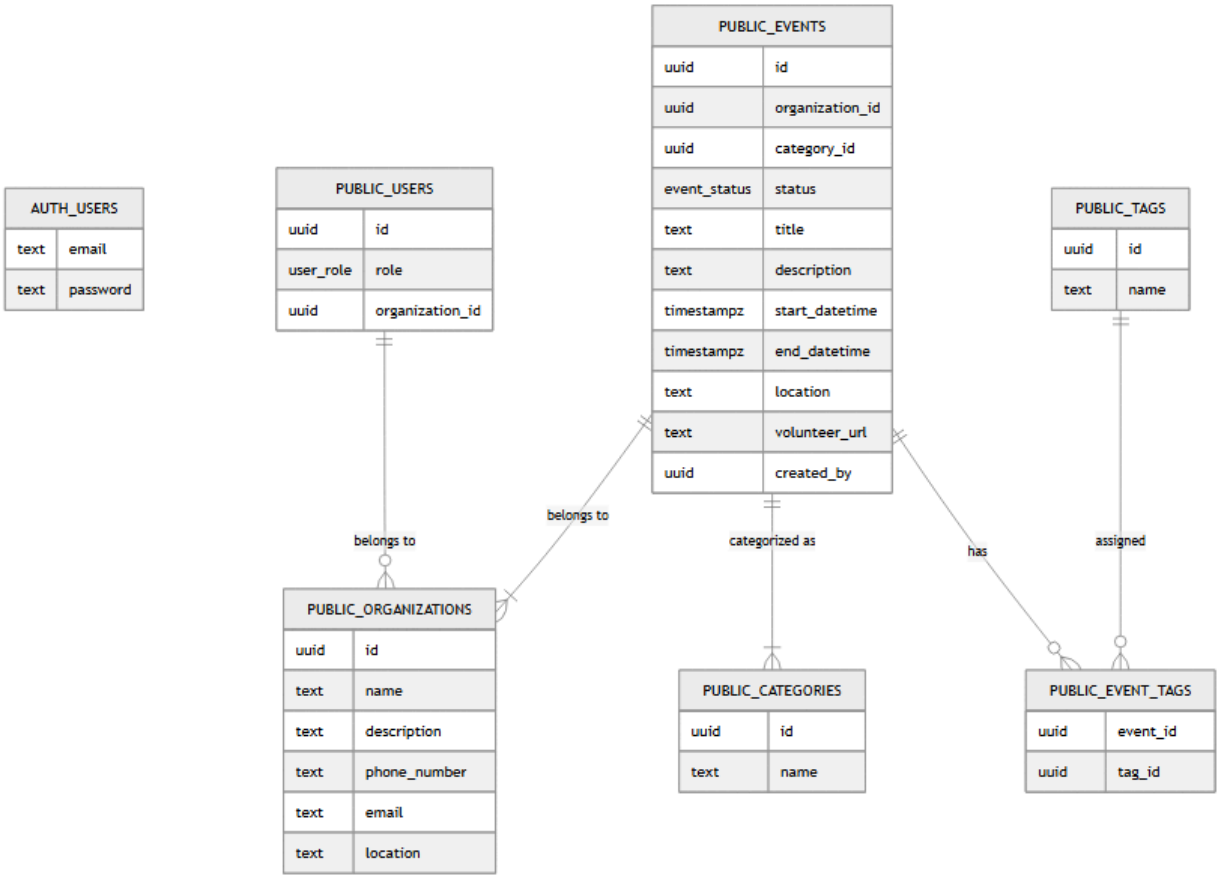

Figure 3: ER Diagram

VII. User Interface Design

Outlines the approach and components used in the platform's interface, ensuring accessibility and ease of navigation for all device types. Describes core screens and accessibility standards.

VII.1. Home Page

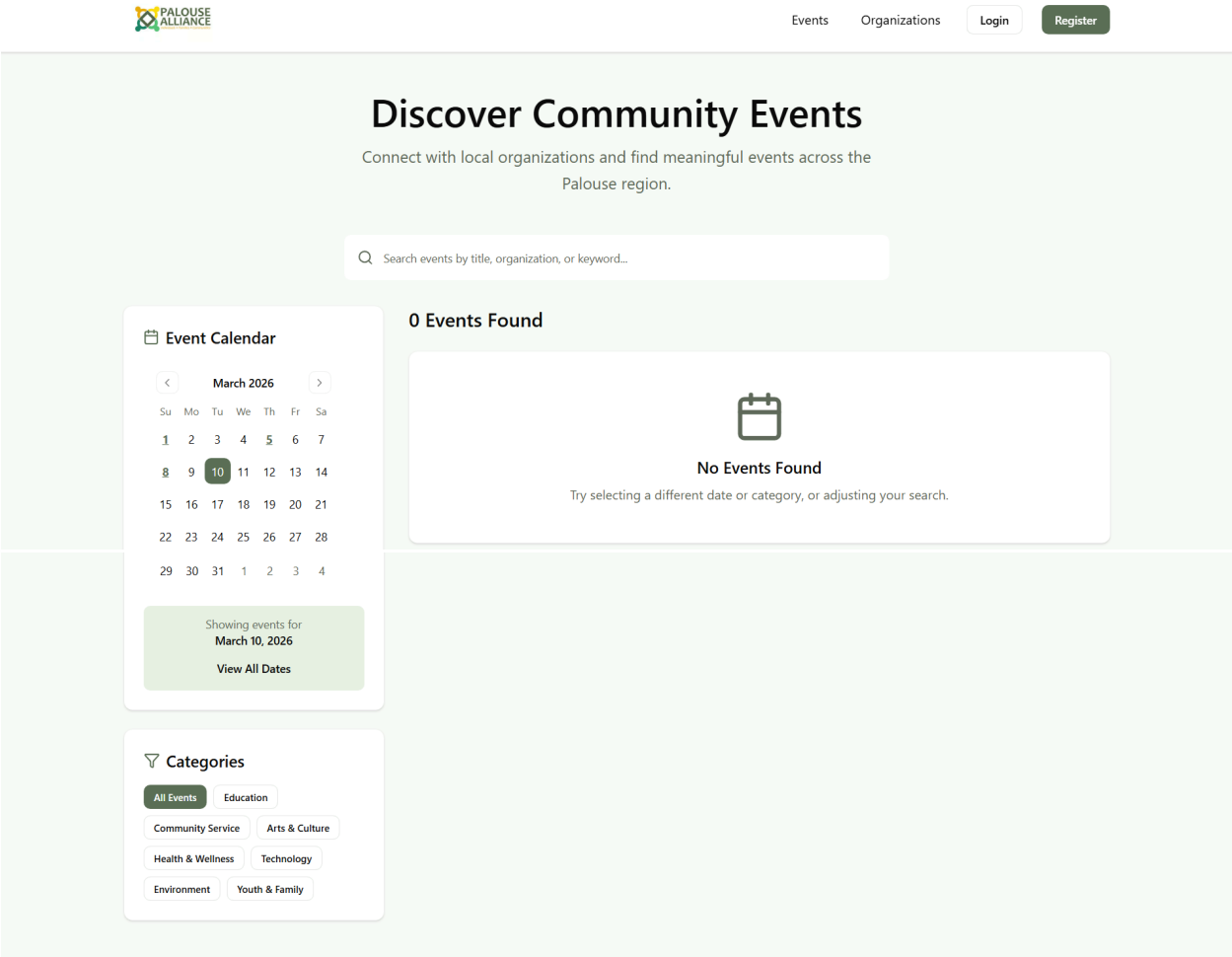

Figure 4: The home page.

Related Functional Requirements:
- FR-1: Search for Services and Events
- FR-2: Category-Based Filtering
- FR-6: View Flyers and Meetings
- FR-14: Responsive Web Design

VII.2. Login Page

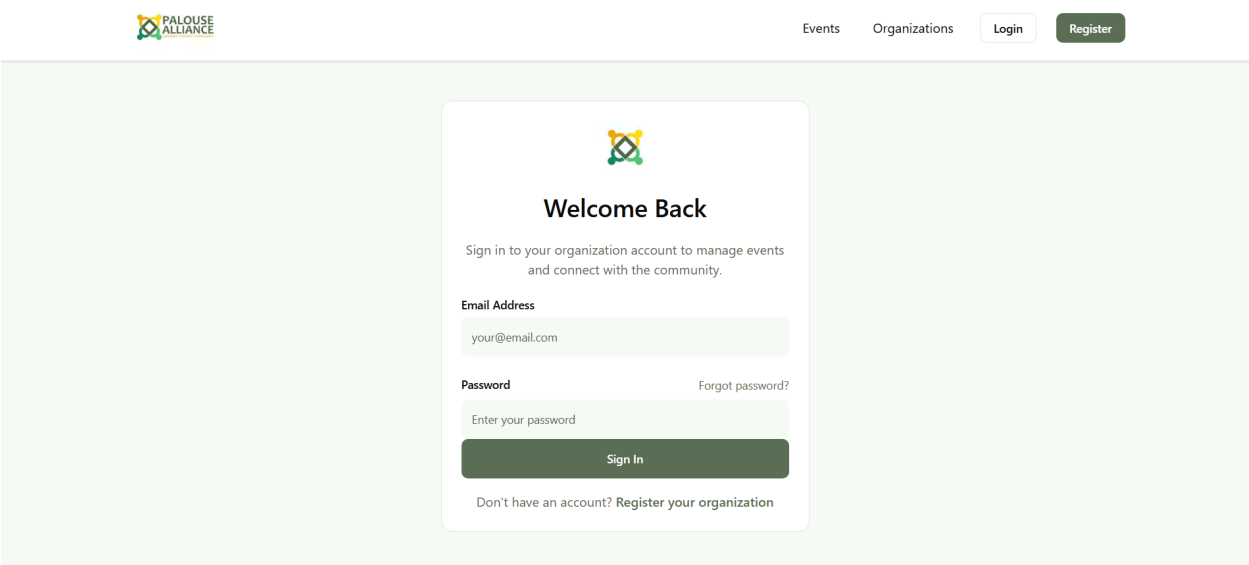

Figure 5: The login page.

Related Functional Requirements:
- FR-4: Member and Admin Account Management
- FR-14: Responsive Web Design

VII.3. Register Page

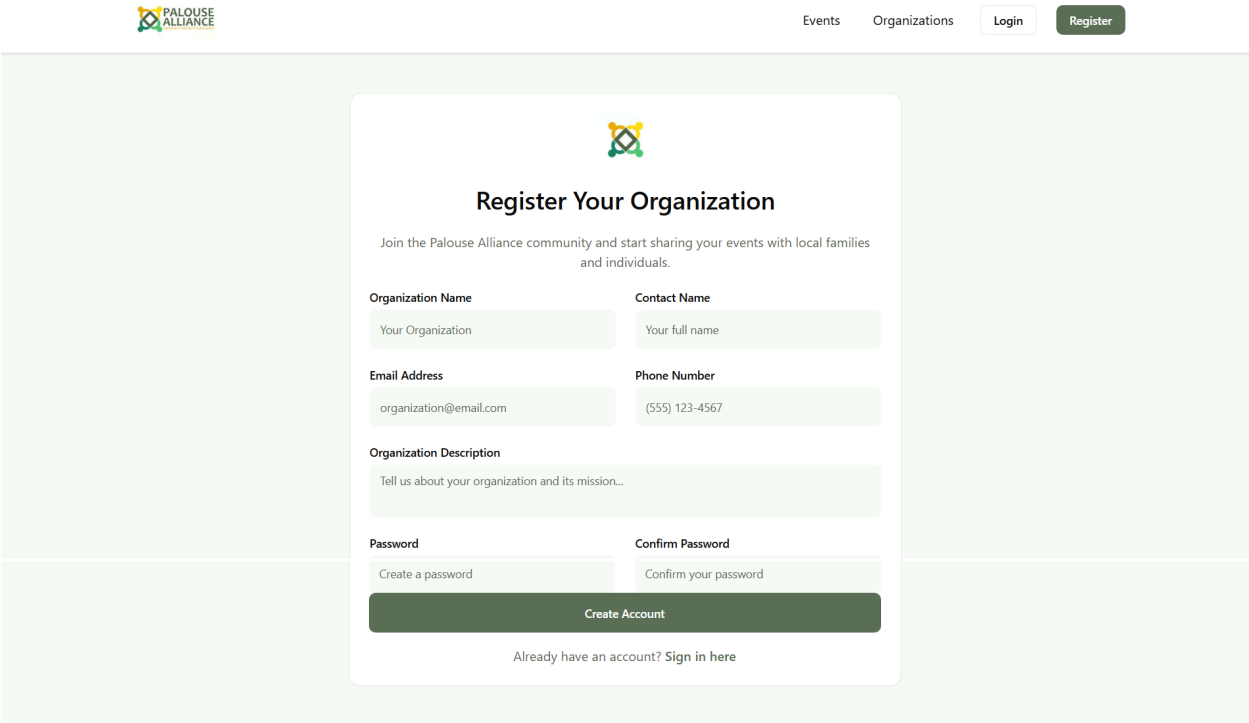

Figure 6: The register page.

Related Functional Requirements:
- FR-4: Member and Admin Account Management

- FR-14: Responsive Web Design

VII.4. Post Event Page

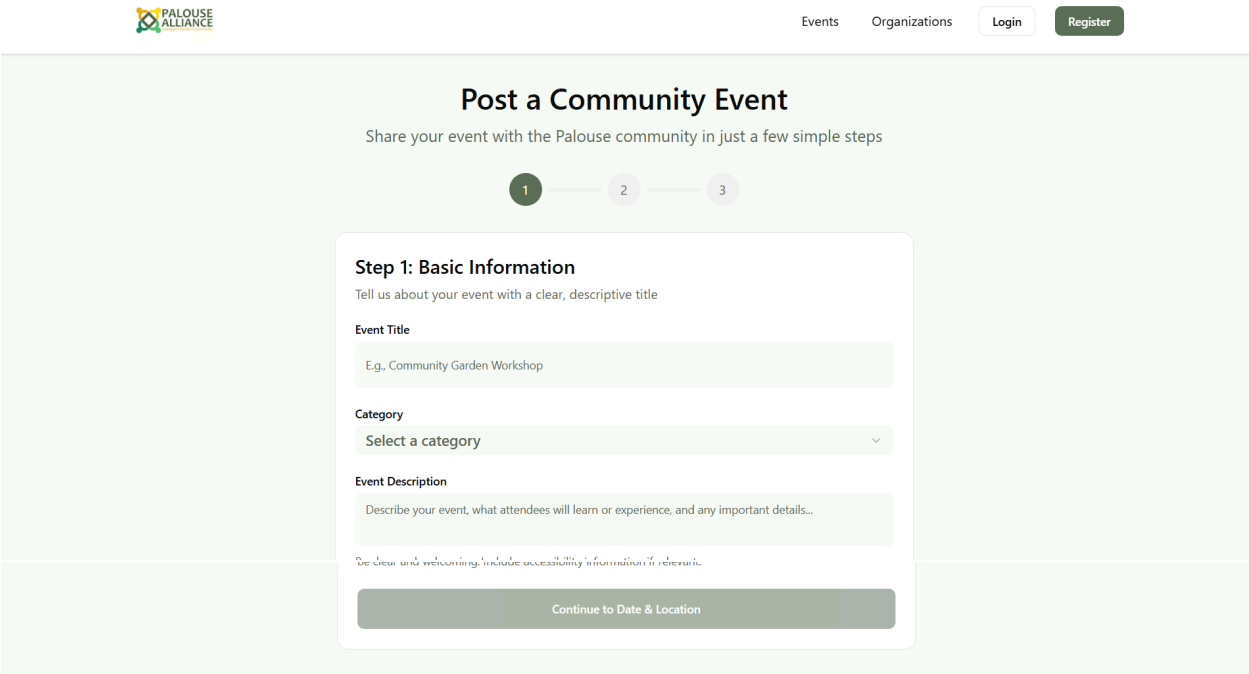

Figure 7: The post event page.

Related Functional Requirements:
- FR-5: Event and Flyer Upload and Approval
- FR-14: Responsive Web Design

VII.5. Dashboard Page

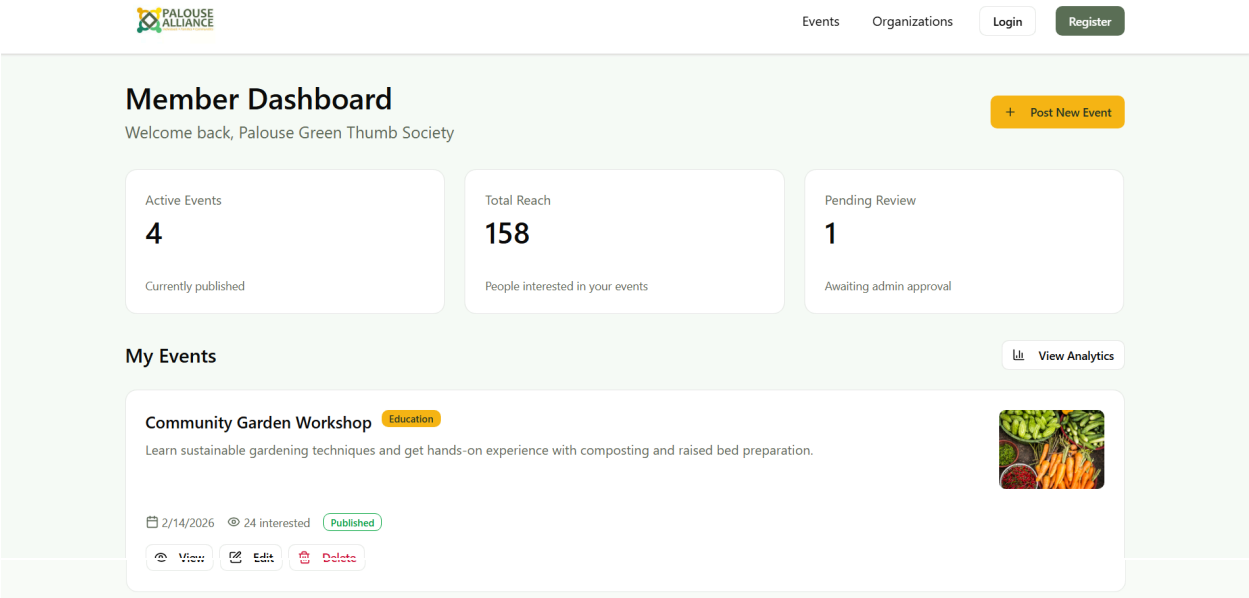

Figure 8: The dashboard page.

Related Functional Requirements:
- FR-6: View Flyers and Meetings
- FR-5: Event and Flyer Upload and Approval
- FR-11: Analytics and Attendance Tracking
- FR-14: Responsive Web Design

VII.6. Admin Page

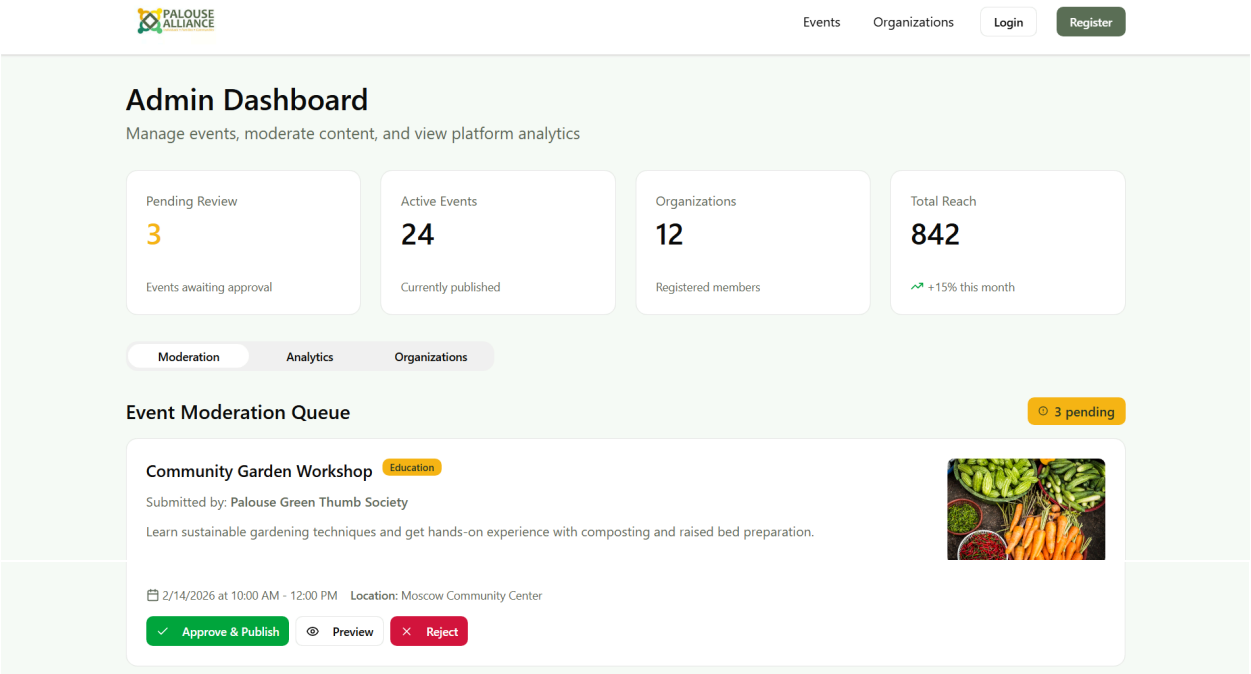

Figure 9: The admin page.

Related Functional Requirements:
- FR-4: Member and Admin Account Management
- FR-5: Event and Flyer Upload and Approval
- FR-11: Analytics and Attendance Tracking
- FR-16: Email Notifications for New Resources
- FR-14: Responsive Web Design

VII.7. Organizations Page

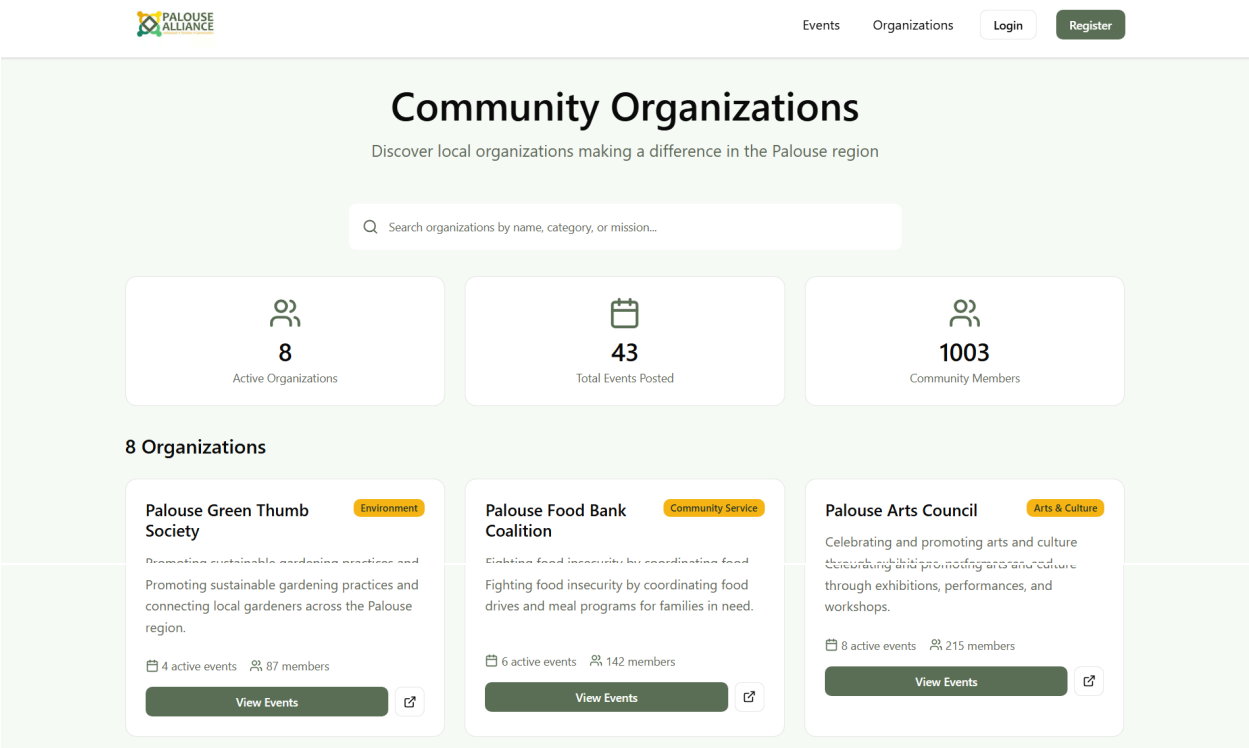

Figure 10: The organizations page.

Related Functional Requirements:
- FR-7: Organization and Agency Listings
- FR-9: Selective Display of Personal Information
- FR-14: Responsive Web Design

VIII. Constraints

Lists the key limitations, requirements, and guidelines that shape the platform's development like accessibility, privacy, scalability, branding, and maintainability. Explains how these constraints affect technical and design decisions.

Category	Description	Example Impact
Accessibility	Must meet **WCAG** 2.2 guidelines.	UI design, screen reader support.
Security	Handle authentication, approval, session management.	Only verified admins approve posts.
Scalability	**Modular architecture** and responsive frontend.	Platform expands for more events.
Privacy	Selective info display and GDPR-style minimization.	Users choose what contact is shown.
Brand Consistency	Follow stakeholder logos and palettes.	All pages use provided colors and logos.
Maintainability	Documented modular codebase and version control.	Easy onboarding for new developers.

IX. Project Testing and Acceptance Plan

IX.1. Overview

This section explains how we will test the Palouse Alliance Community Resource Platform before release. The goal is to make sure the system works as described in the requirements, is easy to use, and behaves reliably for real community members.

Our testing approach is layered. We start with unit tests for small pieces of code, move to integration tests that check how parts work together, and then run full system tests against the deployed application. Finally, we do User Acceptance Testing (**UAT**) with our client and selected Alliance members.

All automated tests that we can reasonably run will live in the GitHub repository and will be executed through **GitHub Actions**.

IX.1.1. Test Objectives and Schedule

The main objectives of our testing work are to confirm that essential features function correctly, to verify the most important non-functional qualities within our project limits, and to keep a clear record of what has been tested. We want to make sure that features like search and filtering, the event calendar, posting and approving events and flyers, managing organization listings, viewing volunteer opportunities, and handling member and admin accounts all work as described in the requirements. We also want to check basic non-functional requirements, such as accessibility on core pages, reasonable security practices, responsiveness on common devices, and acceptable performance for normal use cases. In addition, we aim to store our test code and test results in GitHub so that each release has visible evidence of what was verified.

Our testing schedule follows the project sprints. Early in the project, we will focus on setting up **GitHub Actions**, writing unit tests as we develop features, and adding basic integration tests that can run against a local or shared test setup. In the middle of the project, we will expand integration and end-to-end tests and run them more often on the deployed application. Closer to the end, we will concentrate on broader system testing and **UAT** with the client. Typical activities during this schedule include writing and updating automated tests in the repository, configuring and maintaining **GitHub Actions** workflows, preparing and refreshing test data, running smaller performance and accessibility checks, and collecting feedback from **UAT** sessions. The main milestones are having unit tests running and passing for pull requests, having core integration tests passing for key workflows, completing system tests without any open critical issues, and finishing **UAT** with client approval of core functionality.

IX.1.2. Scope

The scope of this testing and acceptance plan covers the web application code stored in our GitHub repository, which includes the Vite/React frontend, the backend routes and APIs, and interactions with the database through Supabase/PostgreSQL. We intend to test all functional features listed in the requirements, from FR-1 through FR-16, such as searching for events, viewing and posting flyers, managing organizations, and sending email notifications. We will also check non-functional aspects that are realistic for a student project, including basic performance checks, accessibility scans against a few main screens, and simple security hygiene like dependency checks and input validation. All automated unit and integration tests

that we can reasonably implement will be committed to the repository and run through **GitHub Actions**. User Acceptance Testing with the client and Alliance members will be performed in the same environment that we use for development and demonstrations.

Some things are intentionally out of scope for this plan. We will not provision and maintain a completely separate, long-lived Supabase project used only for automated tests and load testing, because that adds cost and complexity that do not fit our constraints. Instead, we will reuse our existing environment where it is safe, and rely on local or containerized test databases when possible. We also will not attempt full enterprise-level performance or penetration testing. Instead, we will run lighter checks that still give useful feedback while staying within our time and resource limits.

IX.2. Testing Strategy

Our testing strategy is to rely heavily on automated tests stored in GitHub and run through **GitHub Actions**, and to add targeted manual testing where full automation is too expensive or complex. We will write unit tests to validate individual frontend components and backend functions in isolation. We will add integration tests that exercise real **HTTP** endpoints and basic database interactions, using our current Supabase project or a controlled local database when that makes sense. We also plan to include a smaller set of end-to-end tests that run against a live instance of the app and walk through important user flows. For non-functional areas, we will use tools like Lighthouse or axe to check accessibility on core pages, and run lighter performance checks to make sure the experience is reasonably fast.

We will use Continuous Integration through **GitHub Actions**. Every pull request and change to the main branch will trigger workflows that install dependencies, run linting, and execute unit and selected integration tests. This setup helps us catch problems early and keeps the code base more stable. We will not set up a full Continuous Delivery pipeline that automatically deploys to a separate Supabase environment for every commit, because that would require more infrastructure and configuration than we can justify for this project. Instead, we will run tests in CI against local services started in the workflow, such as a temporary Postgres container, or against a shared development instance when appropriate. Deployments to our main environment will be done manually or with simple scripts. That is acceptable for our team size and release frequency, and we clearly note that a production system with higher stakes would likely need a more advanced CD setup.

IX.3. Test Plans

This section outlines how we plan to carry out each type of testing. Tests will be committed to the repository when possible so that everything is visible and repeatable. In some areas, like performance or security, we will use lighter versions of standard practices because we do not have the budget or time to maintain separate, large-scale environments. We will explain this where it applies.

IX.3.1. Unit Testing

Unit testing will focus on small, isolated units of code. On the frontend side, this means React components, hooks, and helper functions that can be tested without a full browser or backend. On the backend side, it means pure functions, business logic, and validation rules that do not need a live connection to Supabase. We will use Jest or Vitest for the test runner, along with React Testing Library for component tests. These tests will check that components render

correctly with given props, react properly when users type or click, and display the right validation messages when input is incorrect. Backend unit tests will check that functions return expected results, enforce the right conditions for events and listings, and handle errors in a predictable way. Where backend code usually calls Supabase, we will replace those calls with mocks or stubs so that unit tests run quickly and deterministically.

All unit test files will be stored in the GitHub repository alongside the code they test. Our **GitHub Actions** workflows will run unit tests on every pull request and when changes are pushed to the main branch. If any unit test fails, the pull request will be blocked until the issue is fixed. Because we mock external services like Supabase for unit tests, we do not need any extra databases for this level of testing. This makes unit testing very practical and an important first line of defense against bugs.

IX.3.2. Integration Testing

Integration testing will check that different parts of the system work correctly together. For our project, this means checking how the frontend calls backend APIs, how the backend reads and writes data, and how authentication and roles interact with stored data. We will write integration tests that send actual **HTTP** requests to the backend and assert on the responses. Tools such as Supertest can help for API-only tests, and tools like Playwright or Cypress can be used when we also want to drive the user interface. For example, an integration test might create a new event or flyer and verify that it is stored with a “pending” status, then simulate an admin approving it and check that the event now appears in the approved list or calendar. Another example is updating an organization listing and confirming that a later API call returns the updated data.

In a perfect setup, integration tests would run against a completely separate Supabase project that could be reset freely. Since that is outside of our resources, we will compromise. For tests that only need database behavior and not Supabase specific features, we will use a local Postgres database started inside the **GitHub Actions** workflow through Docker. That database can be seeded and cleared without risk. For behavior that is tightly connected to our actual Supabase project, we will keep integration tests lighter and may run some of them manually in our shared environment, using clearly marked test data. We document this choice to show that, while we understand the ideal approach, we are taking a practical path that still demonstrates integration testing and avoids the cost and complexity of a second cloud database instance.

IX.3.3. System Testing

System testing will treat the whole deployed application as a single system. The purpose is to confirm that, when everything is wired together, the platform supports the main **use cases** and meets basic quality expectations. We will run system tests against the same environment we use for normal development and demonstrations. This typically means our main Supabase project and a deployed frontend. Because we are sharing this environment, we must be careful with data. Wherever possible, we will use clearly labeled test records, and we will clean up or archive those records when tests are finished. We will avoid tests that might delete or corrupt data that is needed for class grading or stakeholder demos.

System testing includes several activities. We will run functional tests that walk through complete user flows from the requirements. We will run lighter performance checks to make

sure common actions are not obviously slow. We will verify that we are following key standards and constraints, such as basic accessibility guidelines and security practices that are realistic for this project. We will also perform User Acceptance Testing with our client, which is described separately. Together, these system level checks help us decide whether the application is ready to be shown and used more broadly.

Functional Testing

Functional system testing will focus on end-to-end scenarios that are most important to users and stakeholders. For example, we will test how a public user visits the home page, enters a search term, applies category or tag filters, and views event details on the calendar. We will test how a member logs in, visits the “post event or flyer” page, submits an event, and sees it appear as pending. We will test how an admin logs in, reviews pending submissions, approves or rejects them, and then checks that approved events show publicly while rejected ones do not. We will also test how a member edits an organization listing, adjusts which contact information is public, and then confirms that the public view matches those choices. Finally, we will check that the volunteer section shows **GivePulse** related content or links so that users can find volunteer opportunities.

Some of these flows will be automated using an end-to-end (E2E) testing tool, with test scripts kept in the repository and run as part of CI if feasible. Other flows, especially those that depend heavily on visual layout or external services, might be tested manually using a checklist. Because we are not running these functional tests against a separate copy of our Supabase project, we will use fake or clearly marked test data and be careful to avoid interfering with any “real” demonstration content.

Performance Testing

Performance testing in this project is intentionally limited. We want to gain some confidence that users will not experience extremely slow responses, but we do not have the capacity to run high-volume load tests in a dedicated environment. Our focus, instead, is on simple measurements of page load times and request latencies under light or moderate test load. We will use tools like browser developer tools and Lighthouse to measure how quickly key pages, such as the home page, calendar, and posting forms, render and become interactive.

If time allows, we may also write a small script using **k6** or a similar tool to send a modest number of requests per second to the non-production environment, just to see how the system behaves under a bit of extra pressure. These tests will be kept deliberately small so that they do not overload our shared Supabase project. We will not run prolonged stress tests or high-volume scenarios. We document this limitation clearly so that it is understood that our performance testing is designed for a class project with limited infrastructure, not for full production validation.

Standards and Constraints Verification / Testing

Standards and constraints verification will check that we follow key guidelines from our requirements and solution approach, while recognizing what we can realistically do. For

accessibility, we will use automated tools like axe and Lighthouse on main pages such as the home page, events calendar, login, registration, and posting forms. We will also perform simple manual checks, such as trying to use the site with a keyboard only and doing at least one short screen reader test. The goal is to align with **WCAG**-style practices on the major paths, even if we do not perform a full professional accessibility audit.

For security and privacy, we will rely on GitHub's built-in dependency scanning tools, such as **Dependabot**, to flag known vulnerabilities in our libraries. We will also depend on linting and code review to catch obvious issues like unsanitized user input or insecure patterns. We will verify that our deployed environment uses **HTTPS** and that authentication flows work as expected. We will not run a full penetration test or maintain a separate hardened security environment. Instead, we will choose lightweight tools that integrate with our GitHub repository and avoid extra infrastructure. Finally, we will confirm that branding and other constraints, like using the requested colors and logos and keeping the code base relatively modular and documented, are respected, mainly through review and walkthroughs rather than automated tools.

User Acceptance Testing

User Acceptance Testing will be carried out near the end of the project with our client and a few Alliance members. The purpose is to see whether the system, as delivered, actually supports their real-world tasks. We will prepare short written tasks that map directly to the requirements. For instance, one task might ask a user to find a food assistance event for the current month and note the contact details. Another task might ask a member to log in, post an event, and confirm that it appears after an admin approves it. Yet another task might ask a member to update their organization listing and hide their phone number from the public.

We will run **UAT** using the same environment we use for demos and development, which is backed by our main Supabase project. Because this is not a separate test-only database, we will clearly label test entries and coordinate with the client so they know which records are just for testing. After **UAT** is complete, we will remove or archive the test data. During **UAT** sessions, participants will record any issues, confusing spots, or suggestions. The team will log this feedback in GitHub Issues, fix critical or major problems, and then ask the client for a final sign-off. **UAT** will be considered successful when the client confirms that core tasks can be completed and any remaining issues are minor or cosmetic.

IX.4. Environment Requirements

Our environment setup is built around what is realistic for a GitHub-based student project. Locally, developers will use Node.js with npm or yarn to run the Vite/React frontend and any backend components. Docker and Docker Compose are available so that a local Postgres database or other services can be started easily, which is helpful for some integration tests. Environment variables will be stored in local .env files (not committed to the repository) to connect to Supabase or to a local test database. Developers should be able to run unit tests and some integration tests on their own machines without needing extra cloud resources.

In the CI environment, **GitHub Actions** workflows will install dependencies, run linting, and execute unit and integration tests. When practical, the workflows will start a temporary Postgres container so integration-type tests can run against a clean database that is created and destroyed within the workflow. Any sensitive values, such as API keys or database URLs, will be stored in GitHub Secrets and injected into the workflows as needed.

For shared development, demonstration, system testing, and **UAT**, we will use a single Supabase project and a deployed frontend instance. This environment must support **HTTPS**, use the same basic configuration as our local environment, and include test accounts for different roles like member and admin. We will not create a second Supabase project specifically for testing, because of the setup time and potential cost. Instead, we will manage test data carefully in the shared environment and use local databases when a fully isolated setup is required. We accept that this choice means some types of testing, such as heavy load testing or very destructive database tests, are out of scope.

Test data will be managed through simple seed scripts or manual setup steps where needed. Whenever possible, we will use synthetic or anonymized data instead of real information. The test environment should also support cleaning or resetting data so that we can repeat tests without leftover state causing confusion. By keeping local, CI, and shared environments reasonably aligned, we increase the chances that issues found during testing will match what users experience, and that fixes we apply will hold up once the system is running for the Palouse community.

Glossary

AWS	Amazon Web Services; A cloud computing platform that offers scalable services like storage, databases, and networking operations.
CRUD	Create, Read, Update, Delete; basic database operations.
Dependabot	Automated software dependency management tool associated with GitHub. Helps developers keep their project libraries/packages up-to-date and secure.
GivePulse	Platform designed to help individuals and organizations find and track volunteer opportunities.
GitHub Actions	A continuous integration and continuous deployment platform that allows us to automate software development workflows within our GitHub repository.
HTML	HyperText Markup Language; standard markup language used to create the structure of webpages.
HTTP/HTTPS	HyperText Transfer Protocol; standard, unsecured protocol for sending and receiving data though a web browser.
k6	Open-source testing tool used to test performance, reliability, and API behavior.
Modular Architecture	Design approach that structures a program into independent and interchangeable parts called modules.
SEO	Search Engine Optimization; the process of improving a website to increase its visibility in organic search results, driving unpaid, relevant traffic.
SQL	Structured Query Language; standardized programming language used to manage and manipulate data in databases.
UAT	User acceptance testing. Final phase of testing to determine acceptance criteria.
Use Cases	Description of interactions between an actor and system.
WCAG	Web Content Accessibility Guidelines; standards for making web content accessible.

References

[1] U.S. General Services Administration, “Section508.gov,” [Online]. Available: https://www.section508.gov/. [Accessed: Feb. 15, 2026].

[2] American Public Human Services Association (APHSA), “Why Digital Accessibility Benefits Everyone,” Aug. 06, 2025. [Online]. Available: https://aphsa.org/resources/why-digital-accessibility-benefits-everyone-access-beyond-disabilities-thecatalyst/. [Accessed: Feb. 15, 2026].

[3] W3C Web Accessibility Initiative (WAI), “Web Content Accessibility Guidelines (WCAG) Overview,” [Online]. Available: https://www.w3.org/WAI/standards-guidelines/wcag/. [Accessed: Feb. 15, 2026].

[4] World Wide Web Consortium (W3C), “Web Standards,” [Online]. Available: https://www.w3.org/standards/. [Accessed: Feb. 15, 2026].

[5] W3C, “WAI-ARIA Overview,” [Online]. Available: https://www.w3.org/WAI/standards-guidelines/aria/. [Accessed: Feb. 15, 2026].

[6] U.S. Department of Justice, “ADA Requirements: Effective Communication,” [Online]. Available: https://www.ada.gov/resources/effective-communication/. [Accessed: Feb. 15, 2026].

[7] U.S. Department of Education, Institute of Education Sciences, “ERIC: Education Resources Information Center,” [Online]. Available: https://eric.ed.gov/. [Accessed: Feb. 15, 2026].

[8] IEEE, “IEEE Recommended Practice for Software Requirements Specifications,” IEEE Std 830-1998, 1998.

[9] ISO/IEC/IEEE, “Systems and software engineering, Software life cycle processes,” ISO/IEC/IEEE 12207:2017, 2017.

[10] IEEE, “IEEE Standard for Software Design Descriptions,” IEEE Std 1016-2009, 2009.

[11] IEEE, “IEEE Standard for Software Test Documentation,” IEEE Std 829-2008, 2008.

[12] W3C, “Web Content Accessibility Guidelines (WCAG) 2.2,” W3C Recommendation, 2023.

[13] W3C, “WAI-ARIA 1.2, Accessible Rich Internet Applications,” W3C Recommendation, 2021.

[14] OWASP, “Application Security Verification Standard (ASVS) v4.0.3,” OWASP Foundation, 2021.

[15] T. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” IETF RFC 8446, Aug. 2018.

[16] European Parliament and Council, “Regulation (EU) 2016/679, General Data Protection Regulation (GDPR),” 2016.

[17] U.S. Congress, “Controlling the Assault of Non-Solicited Pornography And Marketing Act of 2003 (CAN-SPAM Act),” 2003.

[18] Lucid Software Inc., “ER Diagram Symbols and Notation,” Lucidchart Blog, [Online]. Available: https://www.lucidchart.com/blog/er-diagram-symbols-and-notation.

Appendix

A. Additional Use Cases

A.1. [UC-6] Manage Deployment and Branding

Use Case	Manage Deployment and Branding
Actors	Admin, Technical Maintainer
Pre Condition	Admin/tech maintainer is authenticated and has access to platform settings.
Post Condition	Custom domain, branding, and **SEO** options updated.
Main Flow	- Admin navigates to deployment/configuration settings. - Admin updates domain name, branding theme, or **SEO** metadata. - The platform reflects the updates appropriately.
Alternative Flow	- Invalid values, system prompts user for correction.
Related Requirements	FR-12: Custom Domain Name Support FR-13: Branding and Theming FR-15: Search Engine Display

B. Additional User Stories

B.1. [US-8] Update Platform Configuration

As an admin or technical maintainer,
I want to update branding and **SEO** settings
so that the platform aligns with our organization's needs and is discoverable online.

Feature: Platform Management
Scenario: Update settings
    Given the admin is in settings
    When they update the branding or **SEO**
    Then the platform records and applies the changes

B.2. [US-9] Discover Volunteer Opportunities

As a resident,
I want to browse volunteer opportunities
so that I can find ways to help my community.

Feature: **GivePulse** Integration
Scenario: Viewing available volunteer opportunities
    Given the user is on the volunteer opportunities page
    When they view the list of available opportunities

Then opportunities integrated from **GivePulse** should be displayed

B.3. [US-10] View Platform Analytics

As an admin,
I want to access analytics and attendance tracking
so that I can monitor site usage and event participation.

Feature: Platform Analytics
Scenario: Admin reviews analytics dashboard
Given the admin is logged into the dashboard
When they navigate to the analytics section
Then usage statistics and attendance reports should be displayed